

故障分析 | 从库并行回放死锁问题分析

原创 林靖华 爱可生开源社区 2021-01-04 16:30

收录于合集

#死锁 2 #数据回放 1 #主从复制 3

作者：林靖华

爱可生服务团队成员，负责处理客户在 MySQL 日常运维中遇到的问题；擅长处理备份相关的问题，对数据库相关技术有浓厚的兴趣，喜欢钻研各种问题。

本文来源：原创投稿

* 爱可生开源社区出品，原创内容未经授权不得随意使用，转载请联系小编并注明来源。

一、背景

生产环境有一套 MySQL 集群，架构为一主两从，其中一个从库设置了延迟复制，延迟时间为 1 天。

某天在巡检实例时，发现这个延迟从库延迟时间已经超过 1 天，且延迟不停的在增加，在监控上查看数据库状态是正常的，其他两台实例也没有出现问题。

登录数据库 `show slave status` 查看状态发现 IO 线程和 SQL 线程的状态都是 `YES`，但实际上 SQL 线程已经出现了报错，信息如下（部分信息已省略）：

```
Slave_IO_Running: Yes
Slave_SQL_Running: Yes

Last_Errno: 1205
Last_Error: Coordinator stopped because there were error(s) in the worker(s). The mo

Seconds_Behind_Master: 451836
Last_IO_Errno: 0
Last_IO_Error:
Last_SQL_Errno: 1205
Last_SQL_Error: Coordinator stopped because there were error(s) in the worker(s). The mo

SQL_Delay: 86400
SQL_Remaining_Delay: NULL
Slave_SQL_Running_State: Waiting for workers to exit
```

这时候如果执行 `stop slave`，这个会话会卡住，`kill` 对应的线程也无法停止，只能通过 `kill -9 MySQL server` 来停止。

实例相关参数配置信息如下，数据库版本为 **5.7.31**。

```
slave_parallel_type = LOGICAL_CLOCK
slave_parallel_workers = 16
slave_preserve_commit_order = on
binlog_transaction_dependency_tracking = WRITESET
transaction_isolation = READ-COMMITTED
```

二、问题排查

`show full processlist` 的结果如下：

```
mysql> show full processlist;
+-----+-----+-----+-----+-----+-----+-----+
| Id      | User      | Host      | db  | Command | Time | State |
+-----+-----+-----+-----+-----+-----+-----+
| 75846950 | system user |          | NULL | Connect | 3193 | Waiting for workers to exit |
| 75846951 | system user |          | NULL | Connect | 29352 | Waiting for preceding transaction |
| 75846952 | system user |          | NULL | Connect | 29352 | Waiting for preceding transaction |
| 75846953 | system user |          | NULL | Connect | 29352 | Waiting for preceding transaction |
| 75846954 | system user |          | NULL | Connect | 29352 | Waiting for preceding transaction |
| 75846955 | system user |          | NULL | Connect | 29352 | Waiting for preceding transaction |
| 75846956 | system user |          | NULL | Connect | 29352 | Waiting for preceding transaction |
| 75846957 | system user |          | NULL | Connect | 29352 | Waiting for preceding transaction |
| 75846962 | system user |          | NULL | Connect | 29352 | Waiting for preceding transaction |
| 75846963 | system user |          | NULL | Connect | 29352 | Waiting for preceding transaction |
| 75846964 | system user |          | NULL | Connect | 29352 | Waiting for preceding transaction |
| 75846965 | system user |          | NULL | Connect | 29352 | Waiting for preceding transaction |
| 75846966 | system user |          | NULL | Connect | 29352 | Waiting for preceding transaction |
| 75847163 | system user |          | NULL | Connect | 337164 | Waiting for master to send data |
| 75859848 | admin      | localhost | NULL | Query   | 0 | starting |
+-----+-----+-----+-----+-----+-----+-----+
27 rows in set (0.00 sec)
```

根据报错的信息查看 `replication_applier_status_by_worker` 表的内容（部分信息已省略）：

```
mysql> select * from performance_schema.replication_applier_status_by_worker\G
***** 8. row *****
CHANNEL_NAME:
  WORKER_ID: 8
  THREAD_ID: NULL
SERVICE_STATE: OFF
LAST_SEEN_TRANSACTION: c50d7f49-ef3f-11ea-949c-287b09c1cb73:17860074
LAST_ERROR_NUMBER: 1205
LAST_ERROR_MESSAGE: Worker 8 failed executing transaction 'c50d7f49-ef3f-11ea-949c-287b09c1cb73:17860074'
LAST_ERROR_TIMESTAMP: 2020-11-27 14:02:46
***** 11. row *****
CHANNEL_NAME:
  WORKER_ID: 11
  THREAD_ID: NULL
SERVICE_STATE: OFF
LAST_SEEN_TRANSACTION: c50d7f49-ef3f-11ea-949c-287b09c1cb73:17860077
LAST_ERROR_NUMBER: 1205
LAST_ERROR_MESSAGE: Worker 11 failed executing transaction 'c50d7f49-ef3f-11ea-949c-287b09c1cb73:17860077'
LAST_ERROR_TIMESTAMP: 2020-11-27 14:02:46
***** 12. row *****
CHANNEL_NAME:
  WORKER_ID: 12
  THREAD_ID: NULL
SERVICE_STATE: OFF
LAST_SEEN_TRANSACTION: c50d7f49-ef3f-11ea-949c-287b09c1cb73:17860078
LAST_ERROR_NUMBER: 1205
LAST_ERROR_MESSAGE: Worker 12 failed executing transaction 'c50d7f49-ef3f-11ea-949c-287b09c1cb73:17860078'
LAST_ERROR_TIMESTAMP: 2020-11-27 14:01:55
```

查看报错的 `gtid` 对应的语句，三条均为 `REPLACE INTO tbl_name(col_name, ...) values (...)` 这样的语句，其中有两数据比较接近，但并不是同一行。

在 Google 上查询相关的情况，发现这个问题很大可能跟从库并行回放发生死锁有关，而且还有可能涉及到 MySQL 的 BUG。

在测试环境创建与生产上相似的表，插入一些数据用作测试，来观察下报错语句的加锁情况，表结构如下：

```
mysql> show create table test;
+-----+-----+
| Table | Create Table |
+-----+-----+
| test  | CREATE TABLE `test` (
```

```

`id` int(11) NOT NULL,
`a` varchar(10) NOT NULL,
`b` varchar(10) NOT NULL,
`c` varchar(10) DEFAULT NULL,
`d` varchar(10) DEFAULT NULL,
PRIMARY KEY (`id`,`a`,`b`),
UNIQUE KEY `u_k` (`a`,`b`,`id`),
KEY `i_c` (`c`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 |

```

```

1 row in set (0.00 sec)

```

```

mysql> select * from test;

```

```

+----+-----+-----+-----+-----+
| id | a   | b   | c   | d   |
+----+-----+-----+-----+
| 10 | a10 | b10 | c10 | d10 |
| 20 | a20 | b20 | c20 | d20 |
| 30 | a30 | b30 | c30 | d30 |
| 40 | a40 | b40 | c40 | d40 |
| 50 | a50 | b50 | c50 | d50 |
+----+-----+-----+-----+
5 rows in set (0.00 sec)

```

表上有一个联合主键，一个联合唯一索引和一个普通索引。

执行语句：

```

session 1

begin;
replace into test values(30,'a30','b30','c30','d30');

```

由于这个语句触发了唯一键冲突，在这个情况下会被拆分成 `delete` + `insert` 语句：

```

delete from test where id=30 and a='a30' and b='b30' and c='c30' and d='d30';
insert into test values(30,'a30','b30','c30','d30');

```

`show engine innodb status` 查看目前事务的状态：

```

---TRANSACTION 5568158, ACTIVE 10 sec
4 lock struct(s), heap size 1136, 4 row lock(s), undo log entries 2
MySQL thread id 91634, OS thread handle 139866244536064, query id 38506448 localhost root
TABLE LOCK table `kk`.`test` trx id 5568158 lock mode IX

```

```

RECORD LOCKS space id 33 page no 3 n bits 80 index PRIMARY of table `kk`.`test` trx id 5568158 lo
Record lock, heap no 4 PHYSICAL RECORD: n_fields 7; compact format; info bits 0
 0: len 4; hex 8000001e; asc      ;;
 1: len 3; hex 613330; asc a30;;
 2: len 3; hex 623330; asc b30;;
 3: len 6; hex 00000054f69e; asc    T  ;;
 4: len 7; hex 5a000000510149; asc Z    Q I;;
 5: len 3; hex 633330; asc c30;;
 6: len 3; hex 643330; asc d30;;

RECORD LOCKS space id 33 page no 4 n bits 72 index u_k of table `kk`.`test` trx id 5568158 lock_m
Record lock, heap no 4 PHYSICAL RECORD: n_fields 3; compact format; info bits 0
 0: len 3; hex 613330; asc a30;;
 1: len 3; hex 623330; asc b30;;
 2: len 4; hex 8000001e; asc      ;;

RECORD LOCKS space id 33 page no 4 n bits 72 index u_k of table `kk`.`test` trx id 5568158 lock_m
Record lock, heap no 4 PHYSICAL RECORD: n_fields 3; compact format; info bits 0
 0: len 3; hex 613330; asc a30;;
 1: len 3; hex 623330; asc b30;;
 2: len 4; hex 8000001e; asc      ;;

Record lock, heap no 5 PHYSICAL RECORD: n_fields 3; compact format; info bits 0
 0: len 3; hex 613430; asc a40;;
 1: len 3; hex 623430; asc b40;;
 2: len 4; hex 80000028; asc      (;;

```

以下为这个语句加的锁：

1. 表上的 **IX Lock**
2. 主键上数据 `id=30,a='a30',b='b30'` 的 **X Lock**
3. 唯一索引上数据 `a='a30',b='b30',id=30` 的 **Next-key Lock**
4. 唯一索引上数据 `a='a40',b='b40',id=40` 的 **Next-key Lock**

从上面的加锁情况来看，这个语句涉及了不少的锁，推测可能是由于这个加锁的范围间接导致了死锁的发生。

接下来继续测试在这一行附近执行 `replace into` 语句是否会被堵塞。

我们下面测试四种情况，分别是：

1. `replace into test values(20,'a20','b20','c20','d20');`
2. `replace into test values(25,'a25','b25','c25','d25');`
3. `replace into test values(35,'a35','b35','c35','d35');`
4. `replace into test values(40,'a40','b40','c40','d40');`

分别对应前一行、与前一行之间的间隙、与后一行之间的间隙和后一行。

`replace into` 在没有唯一键冲突的情况下等于 `insert into` 语句

情况一：

被 session 1 的语句堵塞，等待获取唯一索引上数据 `a='a30',b='b30',id=30` 的 `Next-key Lock`

```
---TRANSACTION 5568170, ACTIVE 3 sec inserting
mysql tables in use 1, locked 1
LOCK WAIT 5 lock struct(s), heap size 1136, 4 row lock(s), undo log entries 2
MySQL thread id 91635, OS thread handle 139866245347072, query id 38667777 localhost root update
replace into test values(20,'a20','b20','c20','d20')
----- TRX HAS BEEN WAITING 3 SEC FOR THIS LOCK TO BE GRANTED:
RECORD LOCKS space id 33 page no 4 n bits 72 index u_k of table `kk`.`test` trx id 5568170 lock_m
Record lock, heap no 4 PHYSICAL RECORD: n_fields 3; compact format; info bits 0
 0: len 3; hex 6133330; asc a30;;
 1: len 3; hex 6233330; asc b30;;
 2: len 4; hex 8000001e; asc      ;;
```

情况二：

被 session 1 的语句堵塞，等待获取唯一索引上数据 `a='a30',b='b30',id=30` 前面间隙的 `Insert Intention Lock`

```
---TRANSACTION 5568177, ACTIVE 3 sec inserting
mysql tables in use 1, locked 1
LOCK WAIT 2 lock struct(s), heap size 1136, 1 row lock(s), undo log entries 1
MySQL thread id 91635, OS thread handle 139866245347072, query id 38672698 localhost root update
replace into test values(25,'a25','b25','c25','d25')
----- TRX HAS BEEN WAITING 3 SEC FOR THIS LOCK TO BE GRANTED:
RECORD LOCKS space id 33 page no 4 n bits 72 index u_k of table `kk`.`test` trx id 5568177 lock_m
Record lock, heap no 4 PHYSICAL RECORD: n_fields 3; compact format; info bits 0
 0: len 3; hex 6133330; asc a30;;
 1: len 3; hex 6233330; asc b30;;
 2: len 4; hex 8000001e; asc      ;;
```

情况三：

被 session 1 的语句堵塞，等待获取唯一索引上数据 `a='a40',b='b40',id=40` 前面间隙的 `Insert Intention Lock`

```
---TRANSACTION 5568178, ACTIVE 2 sec inserting
mysql tables in use 1, locked 1
LOCK WAIT 2 lock struct(s), heap size 1136, 1 row lock(s), undo log entries 1
MySQL thread id 91635, OS thread handle 139866245347072, query id 38680995 localhost root update
replace into test values(35,'a35','b35','c35','d35')
----- TRX HAS BEEN WAITING 2 SEC FOR THIS LOCK TO BE GRANTED:
RECORD LOCKS space id 33 page no 4 n bits 72 index u_k of table `kk`.`test` trx id 5568178 lock_m
Record lock, heap no 5 PHYSICAL RECORD: n_fields 3; compact format; info bits 0
 0: len 3; hex 613430; asc a40;;
 1: len 3; hex 623430; asc b40;;
 2: len 4; hex 80000028; asc      (;;
```

情况四：

被 session 1 的语句堵塞，等待获取唯一索引上数据 `a='a40',b='b40',id=40` 的 `X Lock`

```
---TRANSACTION 5568180, ACTIVE 2 sec updating or deleting
mysql tables in use 1, locked 1
LOCK WAIT 3 lock struct(s), heap size 1136, 2 row lock(s), undo log entries 1
MySQL thread id 91635, OS thread handle 139866245347072, query id 38682146 localhost root update
replace into test values(40,'a40','b40','c40','d40')
----- TRX HAS BEEN WAITING 2 SEC FOR THIS LOCK TO BE GRANTED:
RECORD LOCKS space id 33 page no 4 n bits 72 index u_k of table `kk`.`test` trx id 5568180 lock_m
Record lock, heap no 5 PHYSICAL RECORD: n_fields 3; compact format; info bits 0
 0: len 3; hex 613430; asc a40;;
 1: len 3; hex 623430; asc b40;;
 2: len 4; hex 80000028; asc      (;;
```

从上面的结果来看，`replace into` 在遇到唯一键冲突的情况下确实会堵塞相邻行的 `replace into` 操作，那么这个问题是如何导致从库死锁的呢？

这时候就可以结合从库的并行回放机制来讨论，当前主库的参数设置是 `binlog_transaction_dependency_tracking = WRITESET`，在这个设置下，当事务更新的行没有冲突的情况下，是可以并行回放的，即上面的语句都可以并行回放。

为了保证从库上执行事务的顺序与主库一致，从库设置了 `slave_preserve_commit_order = on`，在这种情况下，并行回放上面的语句就有可能会出现死锁。

假设主库并发执行了下面的语句：

```
session 1
replace into test values(30,'a30','b30','c30','d30');
# last_committed=1 sequence_number=2

---

session 2
replace into test values(40,'a40','b40','c40','d40');
# last_committed=1 sequence_number=3

---

session 3
repalce into .....
```

从库回放时先回放了 `session 2` 的事务，由于这个事务的 `sequence_number` 比 `session 1` 大，为了保证提交的顺序，需要等 `session 1` 的事务先 `commit`，但经过上面的测试可以知道，`session 2` 的语句会持有 `id = 40` 这一行的 `Next-key Lock`；而 `session 1` 的语句也需要获取这个锁，所以在这个情况下就导致了死锁。

那么对于 MySQL 来说，发生这种情况，不会自动触发 InnoDB 的死锁检测来回滚事务吗？

实际上确实会回滚事务，但是因为 MySQL 在这个地方有 BUG，在这个调度上可能会导致 `woker` 线程丢失信号，导致整个复制 `hang` 住。

BUG 链接：

- <https://bugs.mysql.com/bug.php?id=87796>
- <https://bugs.mysql.com/bug.php?id=89247>
- <https://bugs.mysql.com/bug.php?id=95249>
- <https://bugs.mysql.com/bug.php?id=99440>

三、疑问解答

1. 为什么在这个一主两从，只有这个延迟从库发生了死锁的问题，另外一个正常复制的从库没有死锁呢？

这个问题可以从从库的回放并发程度来解答，在没有死锁的情况下，正常复制的从库的并发度只有 1-3 左右，而延迟复制从库的并发一直都能跑满 16 线程。

这个是因为对于正常复制的从库来说，每收到一条主库传过来的事务就执行一条，且都能在短时间内提交，没有事务的积压，所以正常情况下并发度并不高；而延迟复制从库，因为设置了 `SQL_DELAY = 86400`，所以会有事务的积压，等 SQL 线程检查 relaylog 里未执行的事务时间超过 86400 秒之后，就会开始回放，由于主库上同时执行的事务超过 16 个，所以延迟复制从库的并行回放能跑满 16 线程，这时候同时执行 `replace into` 的概率就大大增加了，从而增加了死锁的可能。

实际上，后面这个正常复制的从库也出现了死锁的情况。原因是主库 `delete` 大量数据，造成主从延迟，这样的话回放的事务就会堆积，后面从库在追赶主库的 binlog 时也会出现并行回放跑满 16 线程的情况，导致了并发执行 `replace into`。这个现象也证明上面的猜想是正确的。

2. 当前有什么办法可以规避、解决这个问题？

- 如果是类似上面集群的情况，可以尝试调低 `slave_parallel_workers`，前提是调低参数后不会导致主从复制延迟。
- 如果不要从库执行事务顺序与主库一致，可以设置 `slave_preserve_commit_order = off` 来避免死锁的出现。
- 对于业务方来说，可以考虑将 `replace into` 语句改成 `select + insert/update`，判断数据是否存在后再考虑 `insert` 或者 `update`，这样就可以避免 `Next-key Lock` 的出现。
- 官方已经回复这个 BUG 会修复在 **8.0.23**，但 5.7 的修复时间暂未给出。

参考：

<https://dev.mysql.com/doc/refman/5.7/en/replace.html>

<https://zhuanlan.zhihu.com/p/196769001>

<http://mysql.taobao.org/monthly/2015/03/01/>

文章推荐：

技术分享 | 使用备份恢复实例时存在的坑

故障分析 | 正确使用 auth_socket 验证插件

故障分析 | 崩溃恢复巨慢原因分析

社区近期动态



本文关键字：#死锁# #数据回放# #主从复制#

💖 点一下“阅读原文”了解更多资讯

收录于合集 #死锁 2

上一篇 · 故障分析 | 全局读锁一直没有释放，发生了什么？

阅读原文

喜欢此内容的人还喜欢

故障分析 | xtrabackup 多表备份报错“too many open files”
爱可生开源社区



